



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Tarea Práctica #3 Sockets (Equipo)”

Sistemas Distribuidos y Paralelos

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 004

Hora: N5

Salón: 4207

Equipo 9

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
1845517	Andrea Monserrat Méndez Flores	ITS
2109147	Alexia Hernandez Andrade	ITS
2004428	Alejandro Sauqué Pérez	ITS
2077884	Melida Krystal Mendoza Ventura	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 16 de octubre
de 2024

Contents

OBJETIVO	3
DIFICULTADES ENCONTRADAS	3
CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)	4
CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)	6
Conclusiones	7

OBJETIVO

El propósito de este programa es desarrollar un sistema cliente-servidor que permita la transmisión y procesamiento de datos numéricos. El cliente será responsable de generar un archivo con 100,000 números enteros aleatorios en el rango de 1 a 100, los cuales serán enviados al servidor para su procesamiento. El servidor recibirá estos números, calculará el promedio y devolverá el resultado al cliente. Tanto el servidor como el cliente mostrarán en consola el promedio calculado y la respuesta final obtenida.

Este programa tiene como meta ilustrar el uso de sockets para la comunicación en red, así como la manipulación eficiente de grandes conjuntos de datos en un entorno distribuido, combinando el intercambio de información y operaciones de cálculo.

DIFICULTADES ENCONTRADAS

A pesar de contar con conocimientos previos, como equipo enfrentamos varias dificultades durante el desarrollo del programa cliente-servidor. Uno de los principales retos fue la optimización del manejo de grandes volúmenes de datos. Aunque teníamos experiencia en la manipulación de archivos, el procesamiento eficiente de 100,000 números y su transmisión entre cliente y servidor requirió mejoras en la gestión de los datos para evitar problemas de rendimiento. También, la correcta sincronización entre cliente y servidor fue más desafiante de lo que esperábamos.

A pesar de entender los conceptos básicos de sockets, nos encontramos con dificultades para asegurar que ambos lados de la comunicación enviaran y recibieran datos en el momento adecuado, lo que nos llevó a ajustar tiempos de espera y validación de datos.

Otra dificultad importante fue el manejo de excepciones y control de errores. Tuvimos el error de EOFException donde nuestro flujo de datos se estaba cerrando prematuramente o no se está sincronizando bien entre el cliente y el servidor. Al intentar reiniciar el servidor, hubo problemas con puertos ocupados, lo que provocó errores de BindException.

Si bien conocíamos los fundamentos, durante la transmisión de datos surgieron problemas como desconexiones inesperadas o datos incompletos, lo que nos obligó a implementar un manejo más robusto de errores para que el programa pudiera recuperarse eficientemente. Estas dificultades aportaron nuevos aprendizajes y nos impulsó a investigar, lo que finalmente nos permitió completar el proyecto.

CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)

SERVIDOR

```
1 package sockets;
2
3 import java.net.ServerSocket;
4 import java.net.Socket;
5 import java.util.ArrayList;
6 import java.util.List;
7 import java.io.ObjectInputStream;
8 import java.io.ObjectOutputStream;
9 import java.util.Scanner;
10
11 public class Servidor {
12     public static void main(String[] args) throws Exception {
13         ObjectInputStream ois = null;
14         ObjectOutputStream oos = null;
15
16         Socket s = null;
17         ServerSocket ss = new ServerSocket(5433);
18
19         // Pedimos al usuario el número de hilos
20         Scanner scanner = new Scanner(System.in);
21         System.out.print("Introduce el número de hilos (particiones): ");
22         int numHilos = scanner.nextInt();
23
24         System.out.println("Esperando conexiones...");
25
26         while (true) {
27             try {
28                 s = ss.accept();
29
30                 System.out.println("Se conectaron desde la IP: " + s.getInetAddress());
31                 ois = new ObjectInputStream(s.getInputStream());
32                 oos = new ObjectOutputStream(s.getOutputStream());
33
34                 List<Integer> numeros = (List<Integer>) ois.readObject();
35
36                 double promedio = calcularPromedio(numeros, numHilos);
37
38                 oos.writeObject(promedio);
39                 System.out.println("Promedio enviado: " + promedio);
40
41             } catch (Exception ex) {
42                 ex.printStackTrace();
43             } finally {
44                 if (oos != null) oos.close();
45                 if (ois != null) ois.close();
46                 if (s != null) s.close();
47                 System.out.println("Conexión cerrada!");
48             }
49         }
50     }
51
52     private static double calcularPromedio(List<Integer> numeros, int numHilos) throws InterruptedException {
53         int size = numeros.size();
54         int chunkSize = size / numHilos;
55         int remainder = size % numHilos; // Para manejar los residuos al dividir
56
57         List<ThreadPromedio> hilos = new ArrayList<>();
58         int inicio = 0;
59
60         for (int i = 0; i < numHilos; i++) {
61             ThreadPromedio hilo = new ThreadPromedio(numeros, chunkSize, remainder, inicio);
62             hilos.add(hilo);
63             inicio += chunkSize + (i == numHilos - 1 ? remainder : 0);
64         }
65
66         for (ThreadPromedio hilo : hilos) {
67             hilo.start();
68         }
69
70         for (ThreadPromedio hilo : hilos) {
71             hilo.join();
72         }
73
74         double suma = 0;
75         for (ThreadPromedio hilo : hilos) {
76             suma += hilo.promedio;
77         }
78
79         return suma / numHilos;
80     }
81 }
```

```

63
64
65     for (int i = 0; i < numHilos; i++) {
66         int fin = inicio + chunkSize + (i < remainder ? 1 : 0); // Asignar el residuo a las primeras particiones
67         List<Integer> sublista = numeros.subList(inicio, fin);
68         ThreadPromedio hilo = new ThreadPromedio(sublista);
69         hilos.add(hilo);
70         hilo.start();
71         inicio = fin;
72     }
73
74
75     for (ThreadPromedio hilo : hilos) {
76         hilo.join();
77     }
78
79
80     double sumaTotal = 0;
81     for (ThreadPromedio hilo : hilos) {
82         sumaTotal += hilo.getSuma();
83     }
84
85     return sumaTotal / size; // Calcular el promedio total
86 }
87 }
88
89 // Clase que extiende Thread para calcular la suma de una partición
90 class ThreadPromedio extends Thread {
91     private List<Integer> numeros;
92     private double suma = 0;
93
94     public ThreadPromedio(List<Integer> numeros) {
95         this.numeros = numeros;
96     }
97
98     @Override
99     public void run() {
100         for (int numero : numeros) {
101             suma += numero;
102         }
103     }
104
105     public double getSuma() {
106         return suma;
107     }
108 }
109

```

CLIENTE

```

1 package sockets;
2
3 import java.io.BufferedWriter;
4 import java.io.File;
5 import java.io.FileWriter;
6 import java.io.ObjectInputStream;
7 import java.io.ObjectOutputStream;
8 import java.net.Socket;
9 import java.util.ArrayList;
10 import java.util.List;
11 import java.util.Random;
12
13 public class Cliente {
14
15     public static void main(String[] args) throws Exception {
16         ObjectOutputStream oos = null;
17         ObjectInputStream ois = null;
18         Socket s = null;
19
20         try {
21             // Conectarse al servidor
22             s = new Socket("127.0.0.1", 5433);
23             oos = new ObjectOutputStream(s.getOutputStream());
24             ois = new ObjectInputStream(s.getInputStream());
25
26             // Generar lista de números aleatorios
27             List<Integer> numeros = new ArrayList<>();
28             Random random = new Random();
29

```

```

30     for (int i = 0; i < 100000; i++) {
31         int numero = random.nextInt(100) + 1;
32         numeros.add(numero);
33     }
34
35     // Enviar la lista
36     oos.writeObject(numeros);
37     oos.flush();
38
39     // Recibir el promedio
40     Double promedio = (Double) ois.readObject();
41
42     // Mostrar el resultado
43     System.out.println("El promedio calculado por el servidor es: " + promedio);
44 } catch (Exception ex) {
45     ex.printStackTrace();
46 } finally {
47     // Cerrar recursos
48     if (ois != null) ois.close();
49     if (oos != null) oos.close();
50     if (s != null) s.close();
51 }
52 }
53 }
54

```

CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)

SERVIDOR

```

Server para Application: C:\Users\javier\AppData\Local\Programs\Groupbox\
Introduce el número de hilos (particiones): 12
Esperando conexiones...

```

CLIENTE

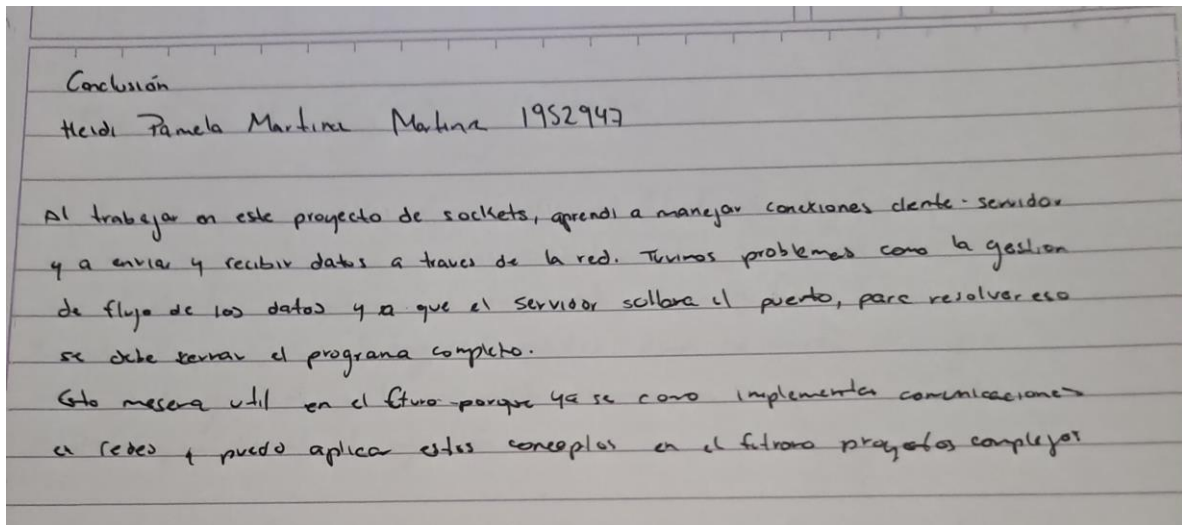
```

Introduce el número de hilos (particiones): 12
Esperando conexiones...
Se conectaron desde la IP: /127.0.0.1
Promedio enviado: 50.6314
Conexión cerrada!

```

Conclusiones

1952947 - Heidi Pamela Martínez Martínez



Andrea Méndez 1845517

Conclusion:

En conclusion, para nosotros esta practica fue un tanto compleja, se consiguió el resultado, pero a diferencia de las anteriores, se podía decir que hubo mucha mas complejidad.

De igual manera debido a la complejidad tuvimos que investigar lo que causó una mayor adquisición de conocimientos en cuanto programación y toma de datos.

Conclusión

Tarea practica #3 Sockets

Alexia Hernández Andrade 2109147

La elaboración de esta tarea practica me aportó nuevos conocimientos y me permitió aplicar y mejorar mis conocimientos previos. Para mi punto de vista fue un poco complicada la conexión y para el uso eficiente de los datos, esto debido a la gran cantidad de datos que se manejan. Además me di cuenta de la importancia de optimizar el manejo de datos ya que era importante transmitirlos y que el programa no se viera afectado, de igual manera fue muy importante estructurar bien el programa para que funcione bajo diferentes condiciones. En conclusión a través de la practica profundizamos conceptos y adquirimos habilidades que nos sirvan para practicas futuras.

Alejandro Sauqué Pérez 2004428 16/10/24
Conclusión Individual Práctica 3

Me parece bastante interesante y útil que aprendamos a usar sockets. En el trabajo hubo una ocasión en la que un compañero y ya tuvimos que usar web sockets para trackear la localidad de un camión, me parece que fue muy enriquecedor aprender de esto ya que es simplemente una manera de permitir que dos equipos se comuniquen. Me gustaría aprender a hacer un socket en Java pero que no solo sea de un dato, sino que sea continuo para mandar estado todo el tiempo. Personalmente, es un tema que sí me interesa genuinamente.

2077884

En conclusión aprendí que los sockets ofrecen una forma viable de la conexión para la comunicación entre distintos procesos y sistemas en una red, permitiendo la transferencia de datos de manera simultánea y optimizada.

Se usaron las conexiones y aunque fue algo complejo y dudoso en las librerías se resolvieron.